

Module 12 - chapter 12

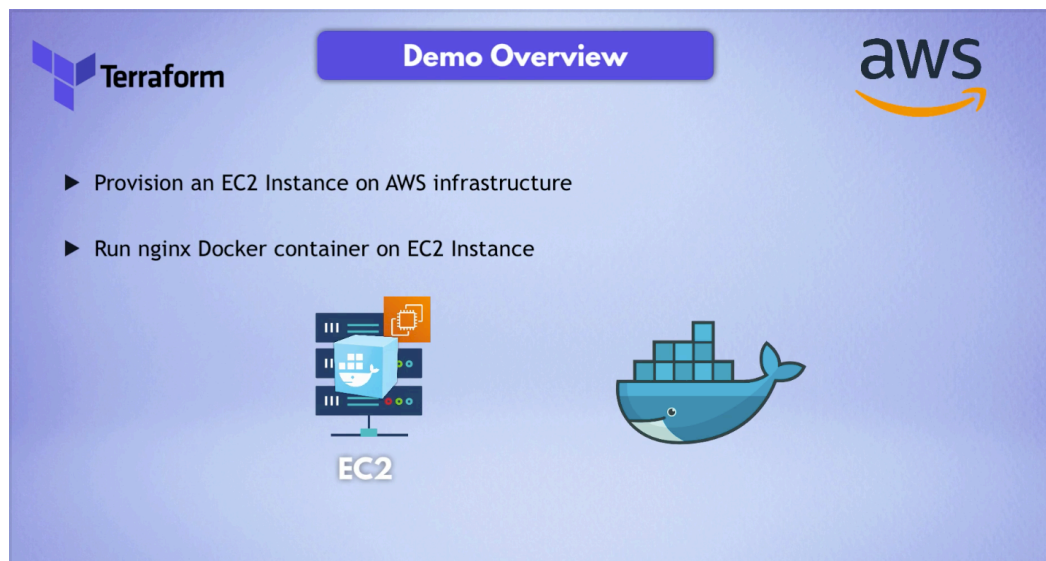
Automate Provisioning EC2 with Terraform - Part 1

The project files used in this lecture can be found here:

Starting code: <https://gitlab.com/twn-devops-bootcamp/latest/12-terraform/terraform-learn>

Demo overview

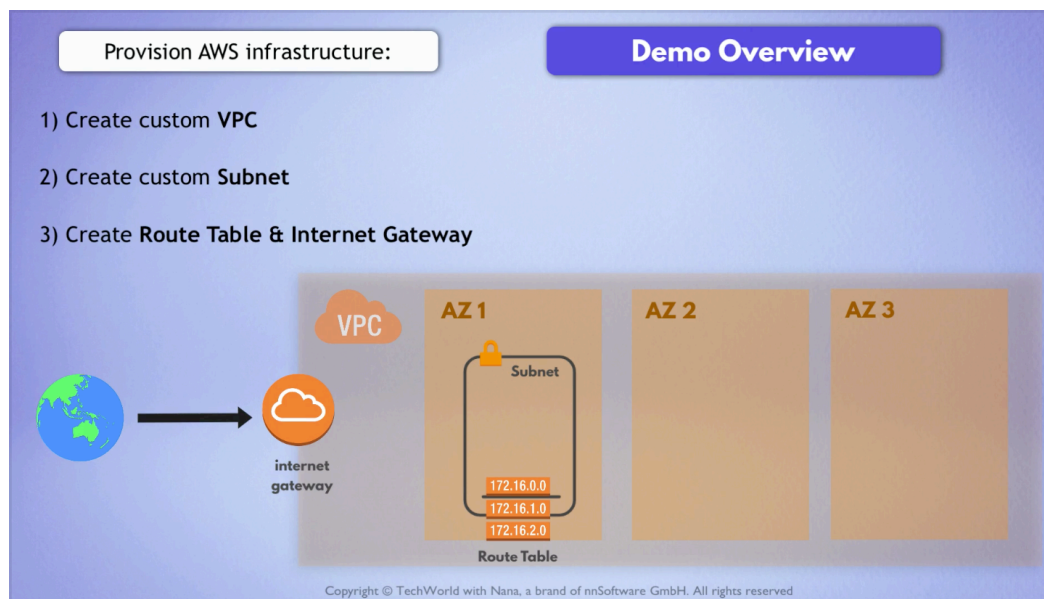
We will deploy an EC2 instance and run a Docker container inside.



We will use Terraform to build the infrastructure on AWS first.

The infra for the demo will have:

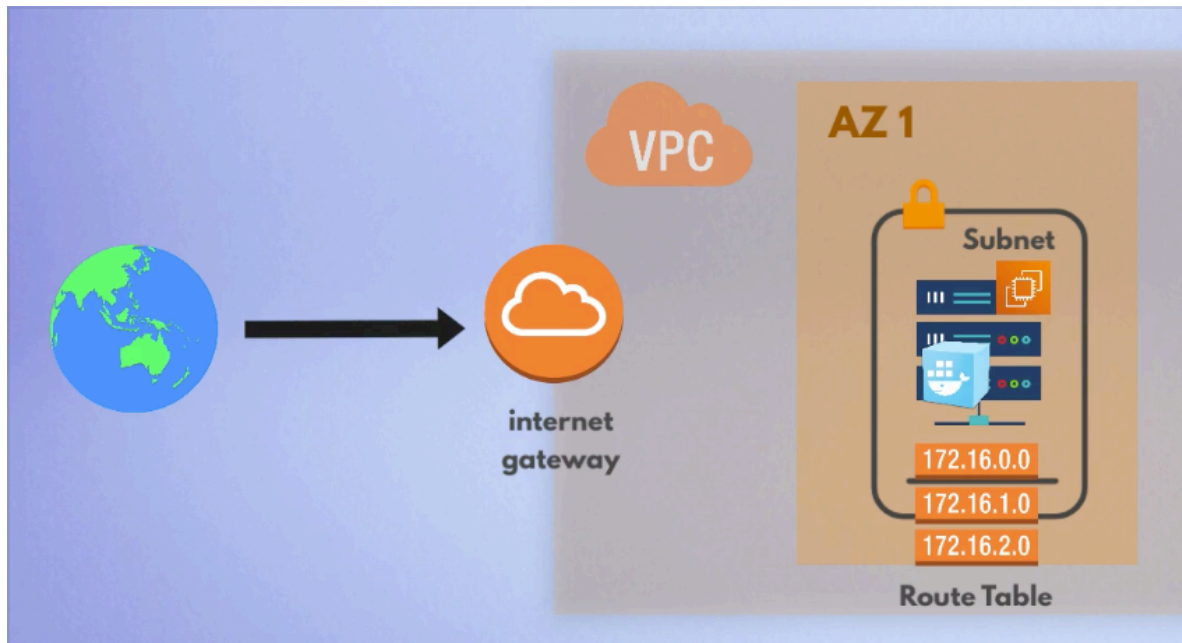
1. VPC
2. One subnet in one AZ (we can have multiple subnets in one AZ but not for this demo)
3. Connect VPC with the internet via Internet gateway



4. Then we will create the EC2 instance
5. Deploy docker container for nginx

4) Provision EC2 Instance

5) Deploy nginx Docker container

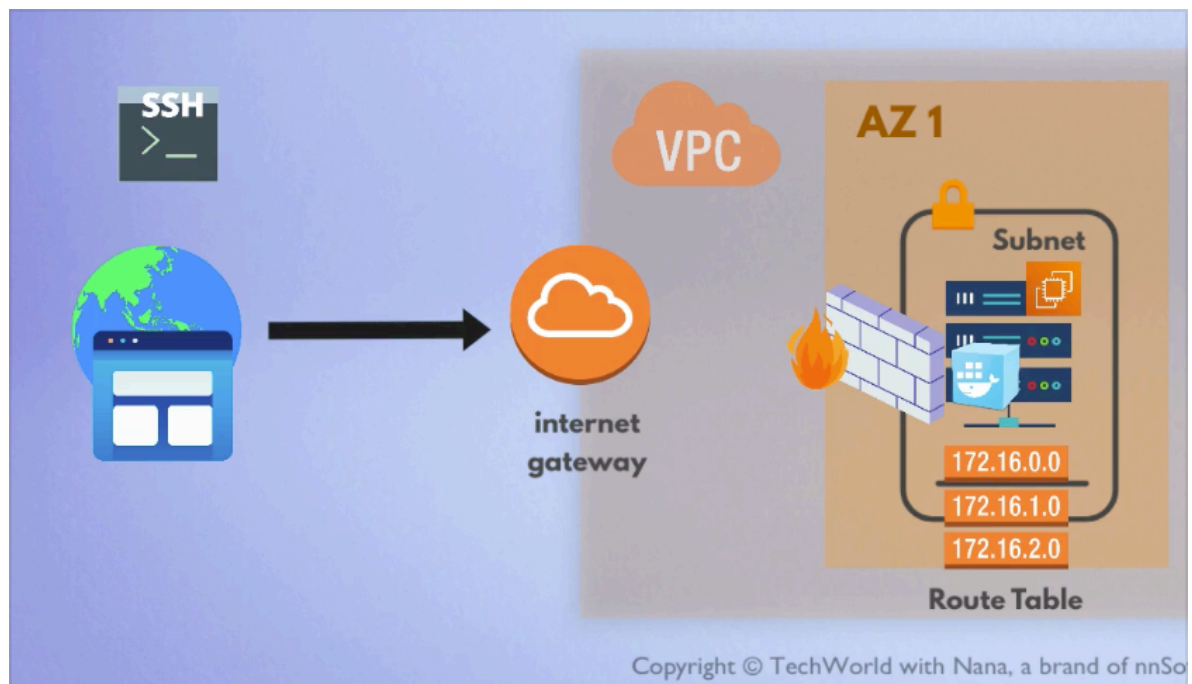


6. Create Security group -> firewall.

Access is:

- from the browser so we need to open firewall rules to be able to access nginx app
- from ssh

6) Create Security Group (Firewall)



Before when we created EC2 instances using AWS console or AWS CLI we were creating the instance inside the default VPC.

Best practice is to create a new VPC first ★

Best practice

- ✓ Create Infrastructure from scratch
- ✓ Leave the default created by AWS as is

VPC and Subnet

1. VPC
2. One subnet in one AZ (we can have multiple subnets in one AZ but not for this demo)

Editing the TF file:

The image shows a code editor with a Terraform configuration file. The code defines variables for VPC CIDR block, subnet CIDR block, availability zone, and environment prefix. It then defines two resources: an AWS VPC and an AWS Subnet. The VPC resource uses the environment prefix for its Name tag. A callout box titled "deployment environment prefix" lists three possible values: "dev-vpc", "staging-vpc", and "prod-vpc".

```
1 provider "aws" {}
2
3 variable vpc_cidr_block {}
4 variable subnet_cidr_block {}
5 variable avail_zone {}
6 variable env_prefix {}
7
8 resource "aws_vpc" "myapp-vpc" {
9   cidr_block = var.vpc_cidr_block
10  tags = {
11    Name: "${env_prefix}-vpc"
12  }
13 }
14
15 resource "aws_subnet" "myapp-subnet-1" {
16   vpc_id = aws_vpc.myapp-vpc.id
17   cidr_block = var.subnet_cidr_block
18   availability_zone = var.avail_zone
19   tags = {
20     Name: var.cidr_blocks[1].name
21   }
22 }
```

deployment environment prefix

- "dev-vpc"
- "staging-vpc"
- "prod-vpc"

TECHWORLD WITH NANA

For naming we use a prefix to tell the environment. We use 'String interpolation' -> \${}

```
main.tf x terraform.tfvars x
1 provider "aws" {}
2
3 variable vpc_cidr_block {}
4 variable subnet_cidr_block {}
5 variable avail_zone {}
6 variable env_prefix {}
7
8 resource "aws_vpc" "myapp-vpc" {
9     cidr_block = var.vpc_cidr_block
10    tags = {
11        Name: "${var.env_prefix}-vpc"
12    }
13 }
14
15 resource "aws_subnet" "myapp-subnet-1" {
16     vpc_id = aws_vpc.myapp-vpc.id
17     cidr_block = var.subnet_cidr_block
18     availability_zone = var.avail_zone
19     tags = {
20         Name: "${var.env_prefix}-subnet-1"
21     }
22 }
23 Pardo, 16/06/2026, 17:24 • TF state and add README
```

And the vars file

```
main.tf x terraform.tfvars x
1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 avail_zone = "eu-west-2a"
4 env_prefix = "dev" |
5
```

-> terraform apply -auto-approve

```

Plan: 2 to add, 0 to change, 0 to destroy.
aws_vpc.myapp-vpc: Creating...
aws_vpc.myapp-vpc: Creation complete after 1s [id=vpc-0635fc1809b770977]
aws_subnet.myapp-subnet-1: Creating...
aws_subnet.myapp-subnet-1: Creation complete after 0s [id=subnet-0ca8d336c9baec8db]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

```

And we check AWS UI

The screenshot shows the AWS VPC console. On the left is a navigation menu with 'Virtual private cloud' expanded. The main panel shows 'Your VPCs' with a table listing two VPCs. The first VPC, 'dev-vpc' (vpc-0635fc1809b770977), is selected. Below the table, the 'Details' tab for 'dev-vpc' is shown, displaying its VPC ID, state (Available), and Block Public Access (Off).

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR	IPv6 CIDR
dev-vpc	vpc-0635fc1809b770977	Available	-	-	Off	10.0.0.0/16	-
-	vpc-03677c88f7cc2f715	Available	-	-	Off	172.31.0.0/16	-

The screenshot shows the AWS VPC console with 'Subnets' selected in the left navigation menu. The main panel shows 'Subnets (1/4)' with a table listing four subnets. The first subnet, 'dev-subnet-1' (subnet-0ca8d336c9baec8db), is selected. Below the table, the 'Details' tab for 'dev-subnet-1' is shown, displaying its Subnet ID, Subnet ARN, state (Available), and Block Public Access (Off).

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 ...	IPv6 ...	Avail...	Availability Zone
-	subnet-03da2ce00b3ba523	Available	vpc-03677c88f7cc2f715	Off	172.31.0.0/20	-	-	4090	euw2-az1 (eu-west-2c)
dev-subnet-1	subnet-0ca8d336c9baec8db	Available	vpc-0635fc1809b770977 dev-vpc	Off	10.0.0.0/24	-	-	251	euw2-az2 (eu-west-2a)
-	subnet-092c85ea013cbc240	Available	vpc-03677c88f7cc2f715	Off	172.31.16.0...	-	-	4091	euw2-az2 (eu-west-2a)
-	subnet-051c96240932f9c7e	Available	vpc-03677c88f7cc2f715	Off	172.31.32.0...	-	-	4090	euw2-az3 (eu-west-2b)

👉 our own custom subnet was added (the other 3 subnets belong to the default VPC)

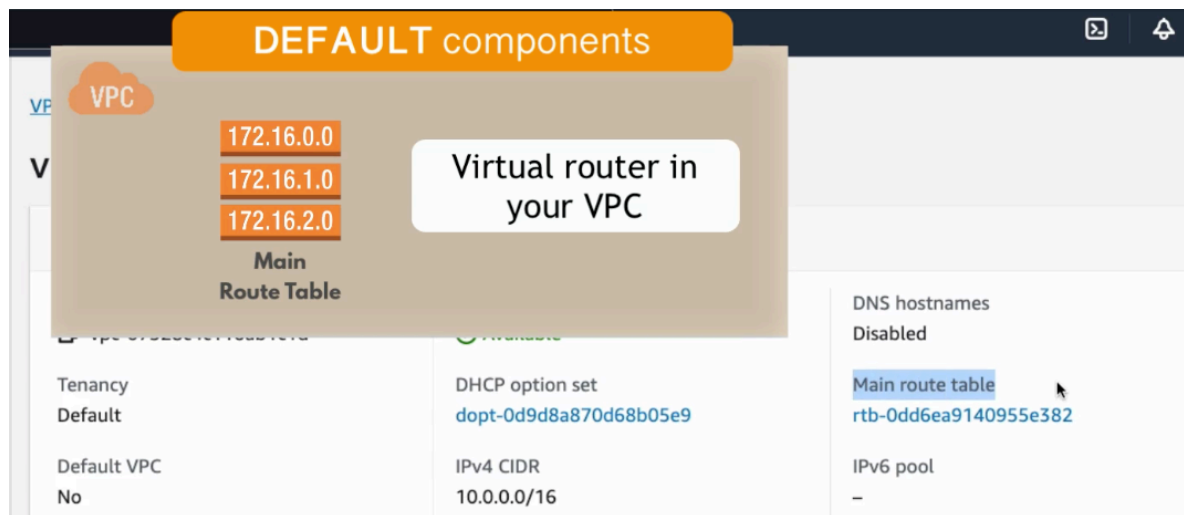
Route Table & Internet Gateway

3. Connect VPC with the internet via Internet gateway

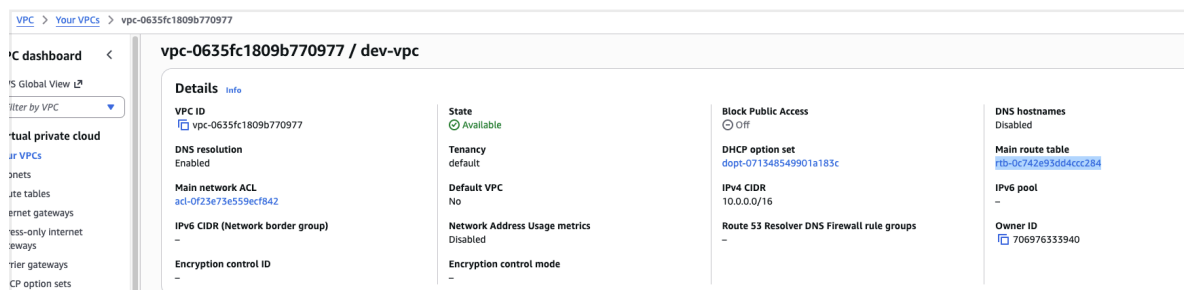
Our New VPC has a Main route table (it was created by default because we didn't specify a route table in TF). A route table is a virtual router inside our private network our VPC.

Our private network has the IP address range of 10.0.0.0/16 available.

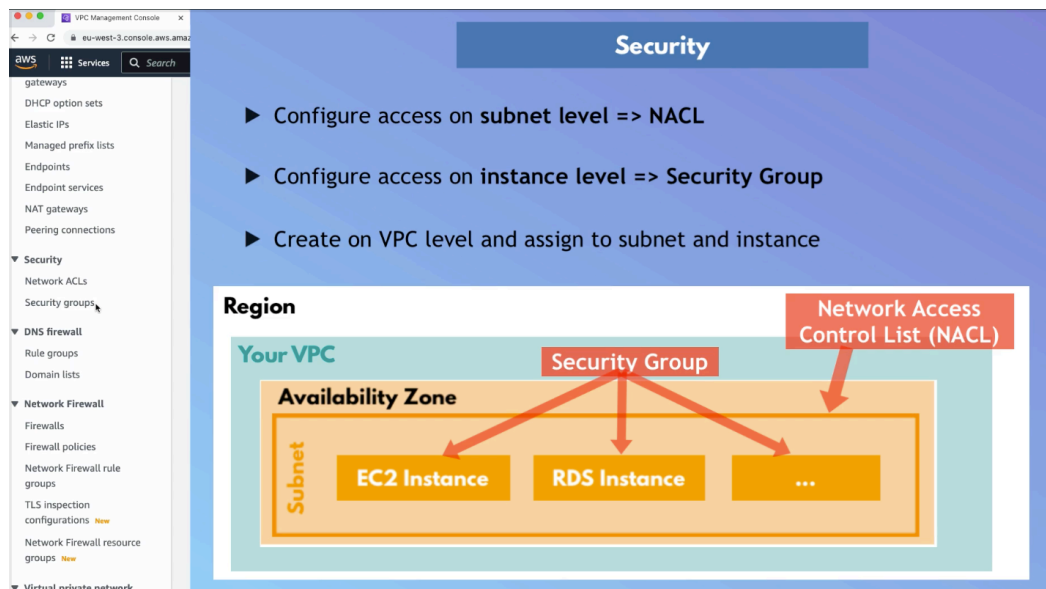
The route table tells where the traffic will be forwarded to within the VPC

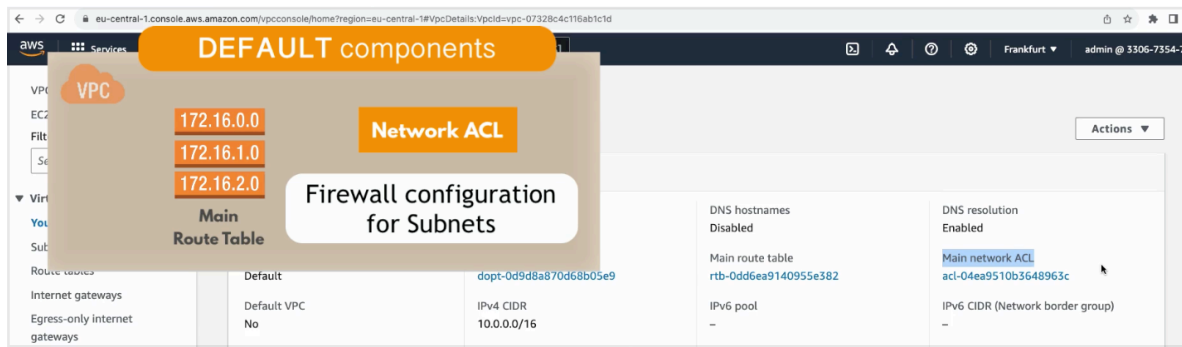


This is from my AWS account

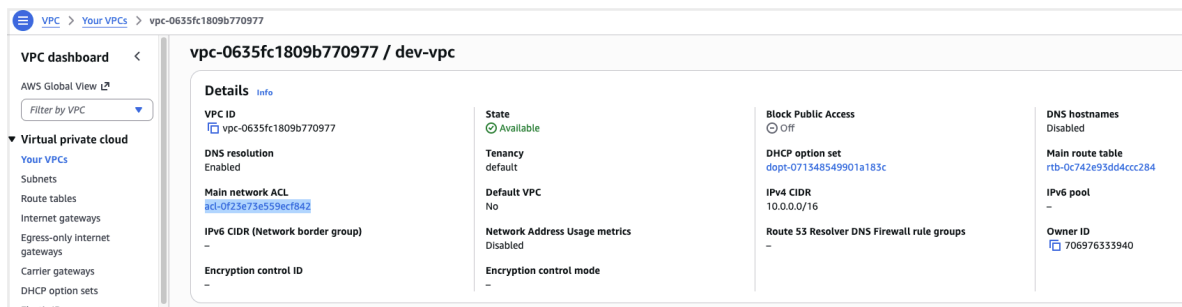


We also have a Network ACL (Access Control List) -> NACL, firewall config for subnets

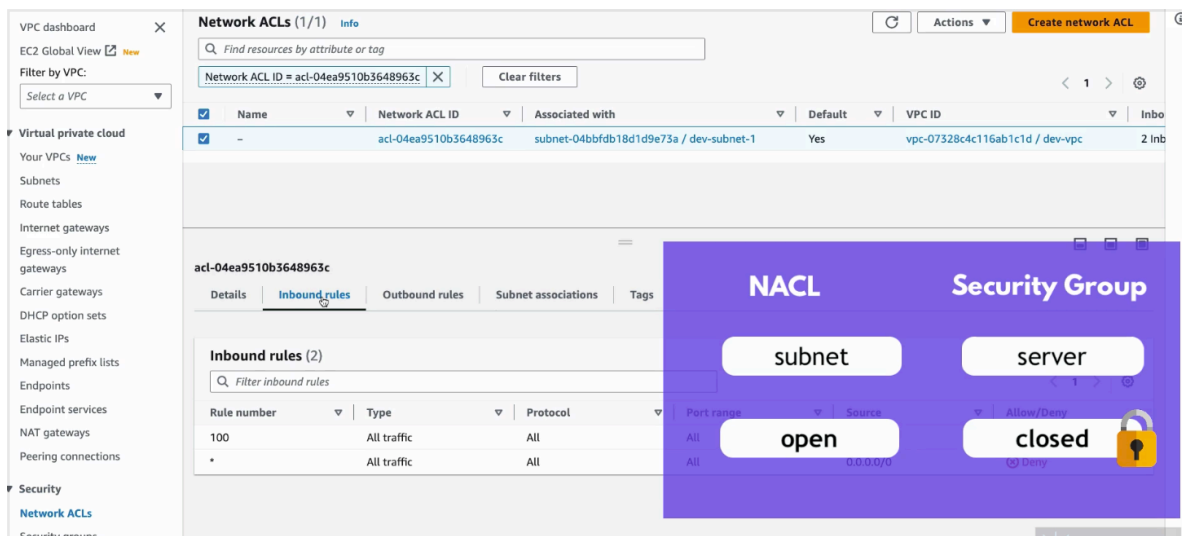




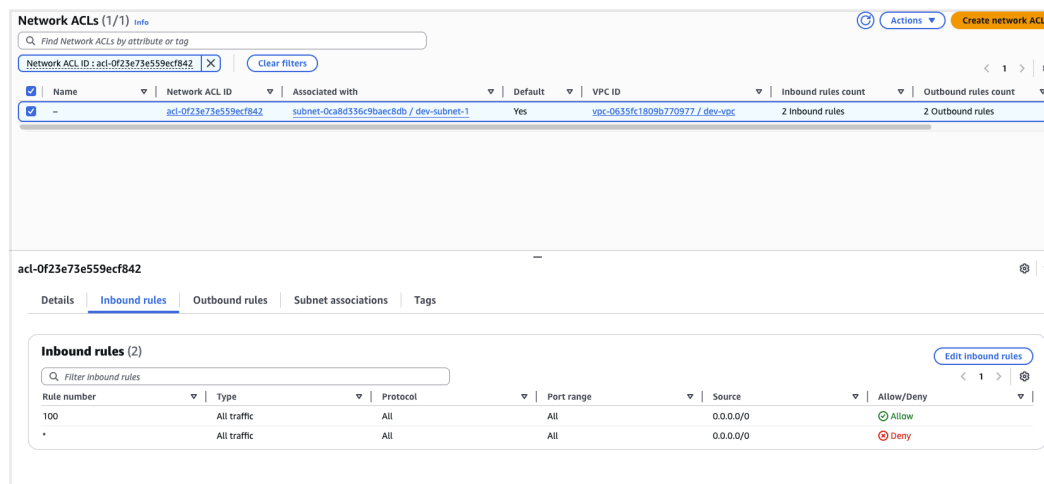
From my AWS account



Taking a look at the NACL



We can see that by default NACL have everything OPEN (Security groups by default are closed) here is my NACL



Name	Network ACL ID	Associated with	Default	VPC ID	Inbound rules count	Outbound rules count
-	acl-0f23e73e559ecf842	subnet-Oca8d536c9baec8db / dev-subnet-1	Yes	vpc-0635fc1809b770977 / dev-vpc	2 Inbound rules	2 Outbound rules

Rule number	Type	Protocol	Port range	Source	Allow/Deny
100	All traffic	All	All	0.0.0.0/0	Allow
*	All traffic	All	All	0.0.0.0/0	Deny

And this is how to read the Inbound Rules:

Rule numbers define the order Terraform evaluates rules — lowest number first. So:

- 100 — first rule evaluated, allows all traffic from anywhere (0.0.0.0/0)
- * — this is the default rule AWS adds automatically to every NACL, it cannot be deleted or modified. It always denies everything that didn't match any previous rule.

How AWS reads this table top to bottom:

1. Does incoming traffic match rule 100? → Yes → Allow, stop evaluating
2. Does incoming traffic match rule 100? → No → fall through to * → Deny

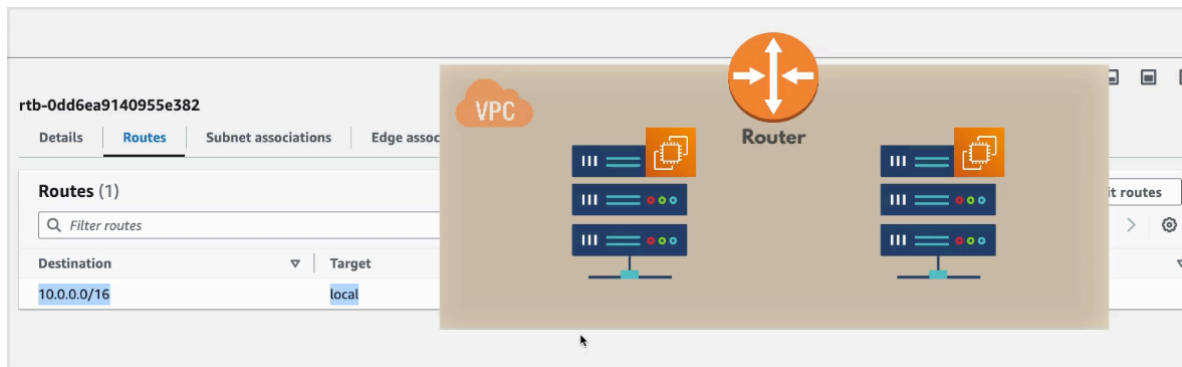
Key things to know about NACLs:

- Rules are evaluated in order, lowest number first
- As soon as a rule matches, AWS stops evaluating — it doesn't continue to the next rule
- The * deny at the end is always there as a safety net
- Rule numbers are typically spaced by 100 (100, 200, 300...) so we can insert new rules in between later if needed

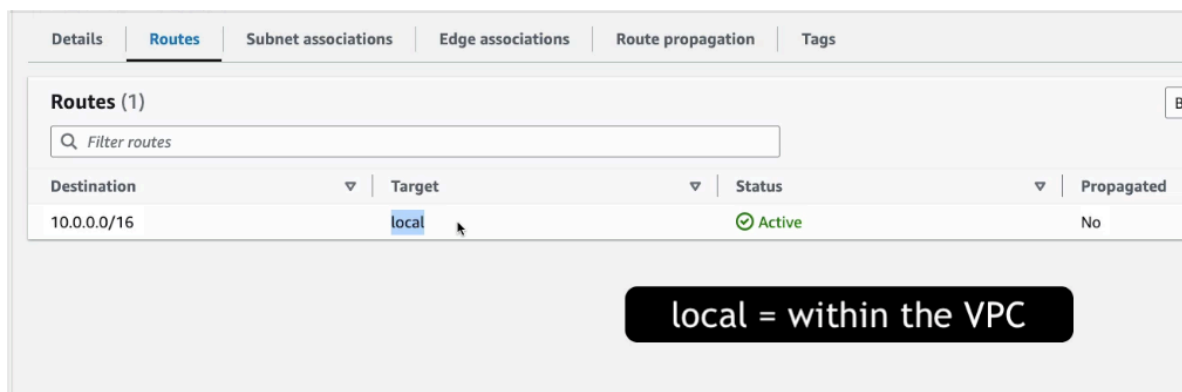
So in our case, rule 100 allows everything in, meaning the * deny will never actually be reached with our current config.

Now let's take a look at the Route table

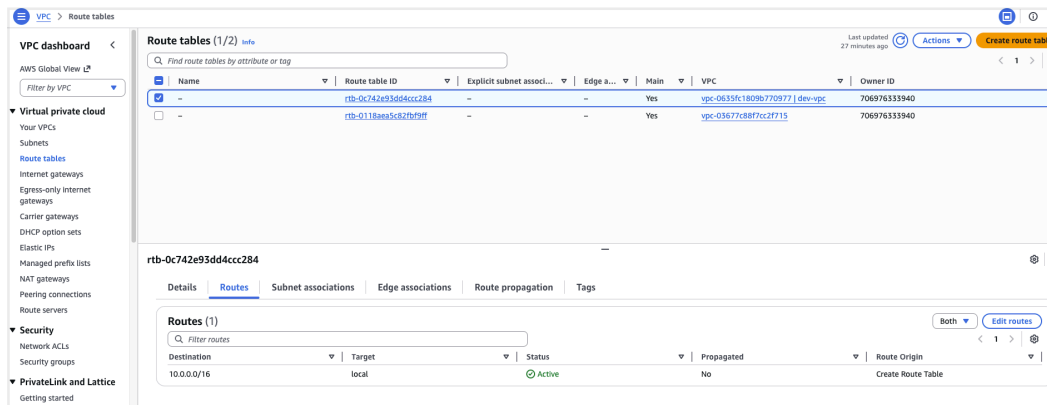
So the Route table handles the traffic inside the VPC for example if we have two EC2 instances



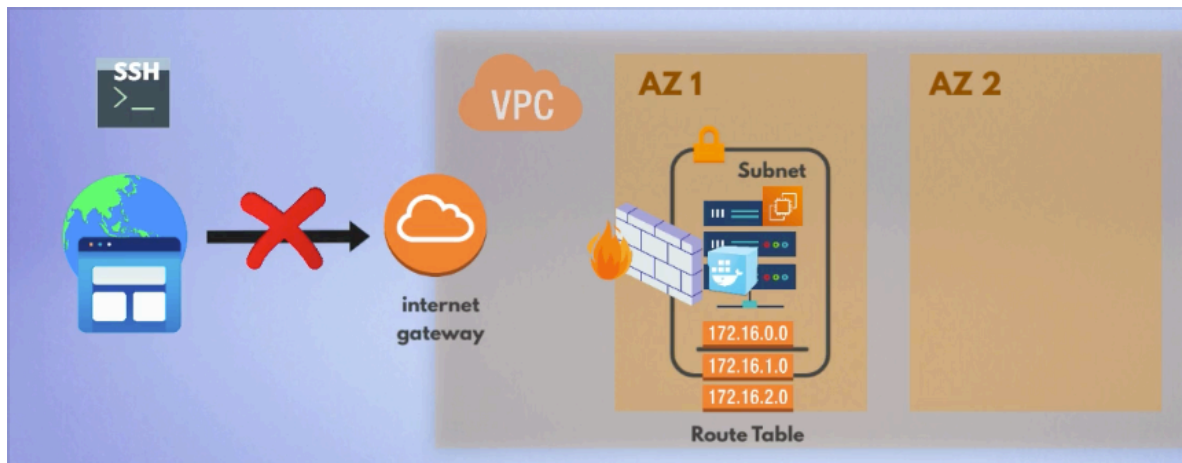
'Local' target -> means within the VPC, it doesn't go out of the VPC.



Here is my Routing table from my AWS account



So no external traffic can access our VPC at the moment



If we compare this route table with the route table of the default VPC

Name	Route table ID	Explicit subnet associ...	Edge a...	Main	VPC	Owner ID
-	rtb-0c742e93dd4ccc284	-	-	Yes	vpc-0635fc1809b770977 dev-vpc	706976333940
-	rtb-0118aea5c82fb9ff	-	-	Yes	vpc-03677c88f7cc2f715	706976333940

rtb-0118aea5c82fb9ff					
Details Routes Subnet associations Edge associations Route propagation Tags					
Routes (2)					
Destination	Target	Status	Propagated	Route Origin	
0.0.0.0/0	igw-01deee367b1559f0	Active	No	Create Route	
172.31.0.0/16	local	Active	No	Create Route Table	

We can see that it has the local target similar to our custom VPC route table but it also has another row another route saying "Any traffic leaving the VPC going to any IP, send it through the Internet Gateway"

The columns:

- Destination — what IP range this route applies to
- Target — where to send the traffic
- Status — Active means the route is working
- Propagated — whether the route was automatically added by a VPN gateway (No = you created it manually)
- Route Origin — how the route was created

Reading your two routes:

Row 1: 0.0.0.0/0 → igw-...

- Any traffic going to the internet (all IPs)
- Send it to the Internet Gateway
- This is what makes your VPC able to reach the internet

Row 2: 172.31.0.0/16 → local

- Any traffic destined for IPs within your VPC range
- Keep it local (stay inside the VPC, don't go to the internet)
- AWS adds this automatically to every route table, it cannot be deleted

How AWS reads it (similar to NACLs):

- Most specific route wins — 172.31.0.0/16 is more specific than 0.0.0.0/0
- So internal traffic stays local, everything else goes to the internet gateway

Note

A route table only controls outbound traffic (where to send traffic leaving the subnet). It doesn't control inbound traffic.

Inbound traffic (coming from the internet) is controlled by:

- Security Groups — at the instance level
- NACLs — at the subnet level

That's why you need both a route table (to allow outgoing) and security group/NACL rules (to allow incoming) for your resources to be fully reachable from the internet.

Once we have the internet Gateway then we can send traffic to internet and it is a requirement to later receive incoming traffic from the internet but we also need to configure the FIREWALL for this.

So the route table with the IGW handles both directions in the sense that the **IGW needs to be in the route table for traffic to flow in either direction.**

"Once we have the Internet Gateway attached to the VPC and a route in the route table pointing to it, traffic can leave to the internet AND the VPC becomes reachable from the internet — but we also need to configure the Security Groups and/or NACLs (the firewalls) to actually allow that inbound traffic through."

The key points:

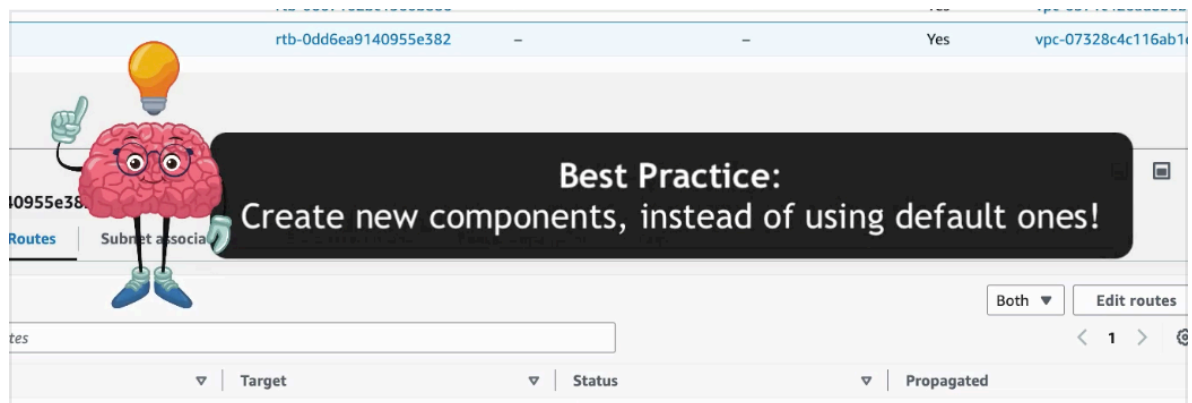
- IGW + route table = necessary foundation for both directions
- Security Groups/NACLs = firewall layer that controls what's actually allowed **in**

A good analogy:

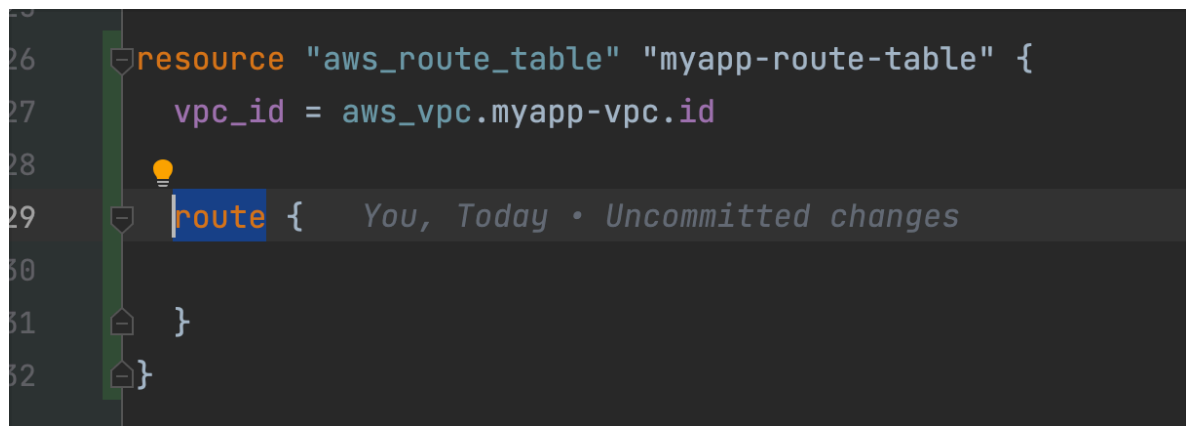
the IGW and route table are like building a road to your house — the road exists in both directions. But the security groups/NACLs are like the door and locks — you still control who can actually come in.

Next, we need to create the Internet Gateway with Terraform 

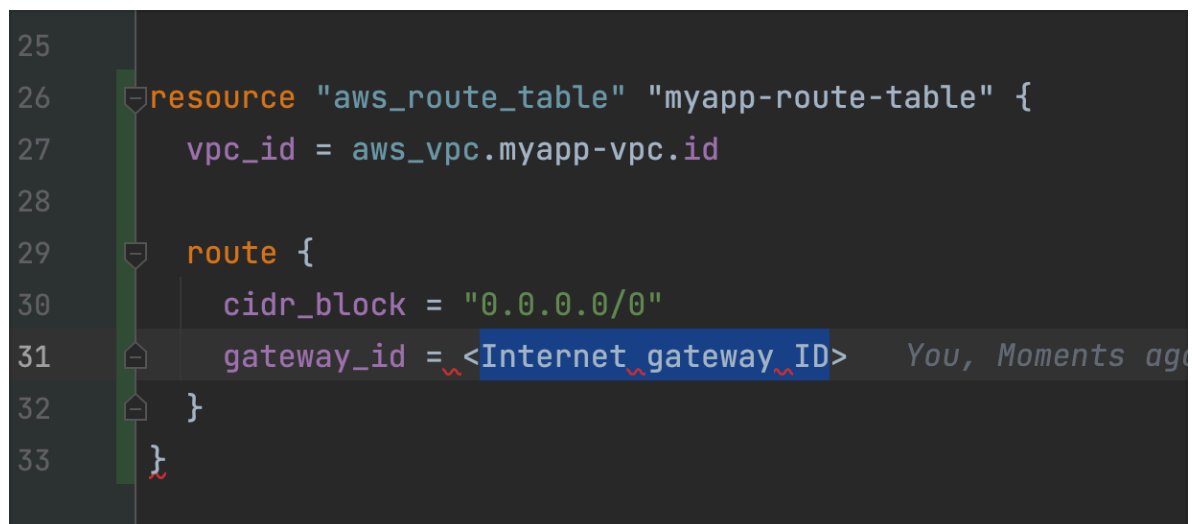
Best practice -> create a new Route Table instead of using a default one



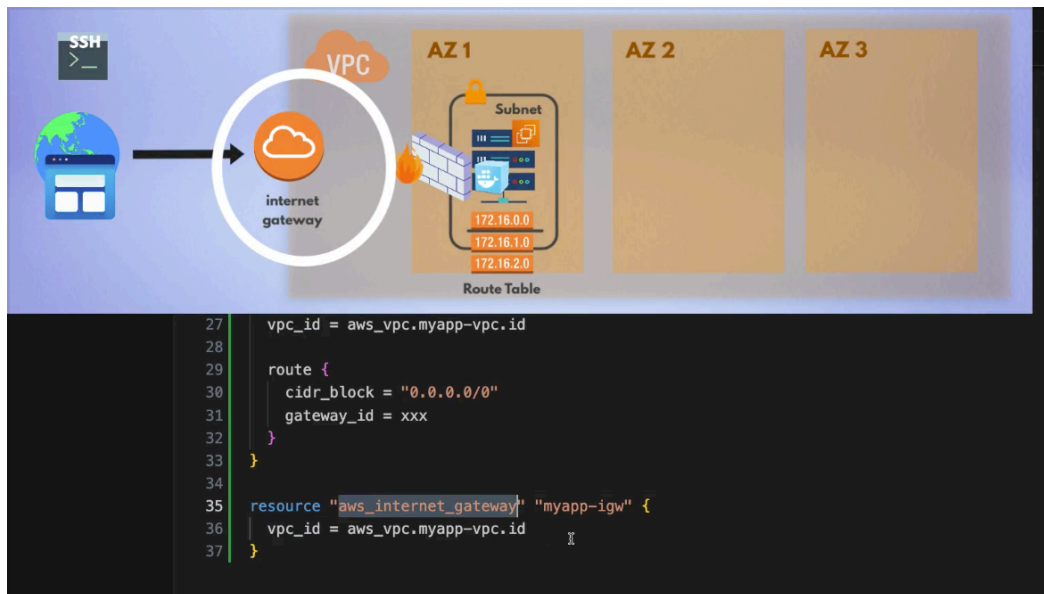
So I create the new resource and attach it to the vpc resource (see line 27)



Now we need to add the two routes, but do we? Not really the route for 'local' target is add it by default so we only need to add the internet gateway - IGW target to the TF configuration



We need an IGW ID, but we don't have an IGW at all -> we need to create one:



And here is the config updated

```
main.tf x terraform.tfvars x
23 }
24 }
25
26 resource "aws_route_table" "myapp-route-table" {
27   vpc_id = aws_vpc.myapp-vpc.id
28
29   route {
30     cidr_block = "0.0.0.0/0"
31     gateway_id = aws_internet_gateway.myapp-igw.id You, Moments ag
32   }
33 }
34
35 resource "aws_internet_gateway" "myapp-igw" {
36   vpc_id = aws_vpc.myapp-vpc.id
37 }
```

And add tags to the resources

```

26 resource "aws_route_table" "myapp-route-table" {
27     vpc_id = aws_vpc.myapp-vpc.id
28
29     route {
30         cidr_block = "0.0.0.0/0"
31         gateway_id = aws_internet_gateway.myapp-igw.id
32     }
33     tags = {
34         Name: "${var.env_prefix}-rtb"
35     }
36 }
37
38 resource "aws_internet_gateway" "myapp-igw" {
39     vpc_id = aws_vpc.myapp-vpc.id
40     tags = {
41         Name: "${var.env_prefix}-igw"
42     }
43 }

```

You, A minute ago • Uncommitted changes

IGW code block can be at the end of the file, TF knows it needs to create it first than the Route table -> TF doesn't care about the order of the declarations.

ar.env_prefix}-subnet-1"

Terraform knows in which sequence the components must be created

internet_gateway myapp-igw []
_vpc.myapp-vpc.id

ar.env_prefix}-igw"



-> terraform plan


```

# aws_internet_gateway.myapp-igw will be created
+ resource "aws_internet_gateway" "myapp-igw" {
  + arn      = (known after apply)
  + id       = (known after apply)
  + owner_id = (known after apply)
  + region   = "eu-west-2"
  + tags     = {
    + "Name" = "dev-igw"
  }
  + tags_all = {
    + "Name" = "dev-igw"
  }
  + vpc_id   = "vpc-0635fc1809b770977"
}

# aws_route_table.myapp-route-table will be created
+ resource "aws_route_table" "myapp-route-table" {
  + arn              = (known after apply)
  + id               = (known after apply)
  + owner_id         = (known after apply)
  + propagating_vgws = (known after apply)
}

```

Two resources will be created

-> terraform apply -auto-approve

```

Plan: 2 to add, 0 to change, 0 to destroy.
aws_internet_gateway.myapp-igw: Creating...
aws_internet_gateway.myapp-igw: Creation complete after 0s [id=igw-00c30f6b1602ed06f]
aws_route_table.myapp-route-table: Creating...
aws_route_table.myapp-route-table: Creation complete after 1s [id=rtb-09e7c96e7e60d6807]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
→ terraform git:(Module_12/Terraform_chapter-12) ×

```

We check our AWS UI

VPC > Route tables

Route tables (1/3) info

Find route tables by attribute or tag

☐	Name	Route table ID	Explicit subnet associ...	Edge a...	Main	VPC	Owner ID
☐	-	rtb-0c742e3d5d4ccc284	-	-	Yes	vpc-06355fc1809b770977 dev-vpc	706976333940
☒	dev-rtb	rtb-09e7c96e7e60d6807	-	-	No	vpc-06355fc1809b770977 dev-vpc	706976333940
☐	-	rtb-0118aea5c82fb9ff	-	-	Yes	vpc-03677c887cc2f715	706976333940

rtb-09e7c96e7e60d6807 / dev-rtb

Details Routes Subnet associations Edge associations Route propagation Tags

Routes (2)

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-0dc309fb1602ad96f	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

And our dev-vpc has now two Route tables, one is a default route table (Main = yes), the new route table is the one we created with TF (main = no) and this one has the local target and the IGW target.

Main = default

Explicit subnet associati...	Edge associations	Main
-	-	Yes
-	-	No
-	-	Yes

our custom one

main = default

And we can see that our new IGW is linked to our VPC

VPC > Internet gateways

VPC dashboard <

AWS Global View

Filter by VPC

Virtual private cloud

- Your VPCs
- Subnets
- Route tables
- Internet gateways
- Egress-only Internet gateways
- Carrier gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers

Security

- Network ACLs
- Security groups

PrivateLink and Lattice

- Getting started

Internet gateways (1/1) [Info](#)

Find internet gateways by attribute or tag

Internet gateway ID: igw-00c30f6b1602ed06f [X](#) [Clear filters](#)

☒	Name	Internet gateway ID	State	VPC ID	Owner
☒	dev-igw	igw-00c30f6b1602ed06f	Attached	vpc-0635fc1809b7709771 dev-vpc	706976333940

igw-00c30f6b1602ed06f / dev-igw

[Details](#) [Tags](#)

Details

Internet gateway ID	State	VPC ID	Owner
igw-00c30f6b1602ed06f	Attached	vpc-0635fc1809b7709771 dev-vpc	706976333940

Subnet Association with Route Table

We need to link subnets with a route table, like we see for the default route table created by TF. It has a 'subnet associations' tab

VPC > Route tables

Route tables (1/3) [Info](#)

Find route tables by attribute or tag

☒	Name	Route table ID	Explicit subnet associ...	Edge a...	Main	VPC	Owner ID
☒	-	rtb-0c742e93dd4ccc284	-	-	Yes	vpc-0635fc1809b7709771 dev-vpc	706976333940
☐	dev-rtb	rtb-09e7c96e7e60d9b807	-	-	No	vpc-0635fc1809b7709771 dev-vpc	706976333940
☐	-	rtb-0118aea5c827bf9ff	-	-	Yes	vpc-03677c887cc27f15	706976333940

rtb-0c742e93dd4ccc284

[Details](#) [Routes](#) [Subnet associations](#) [Edge associations](#) [Route propagation](#) [Tags](#)

Explicit subnet associations (0) [Edit sub](#)

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
No subnet associations You do not have any subnet associations.			

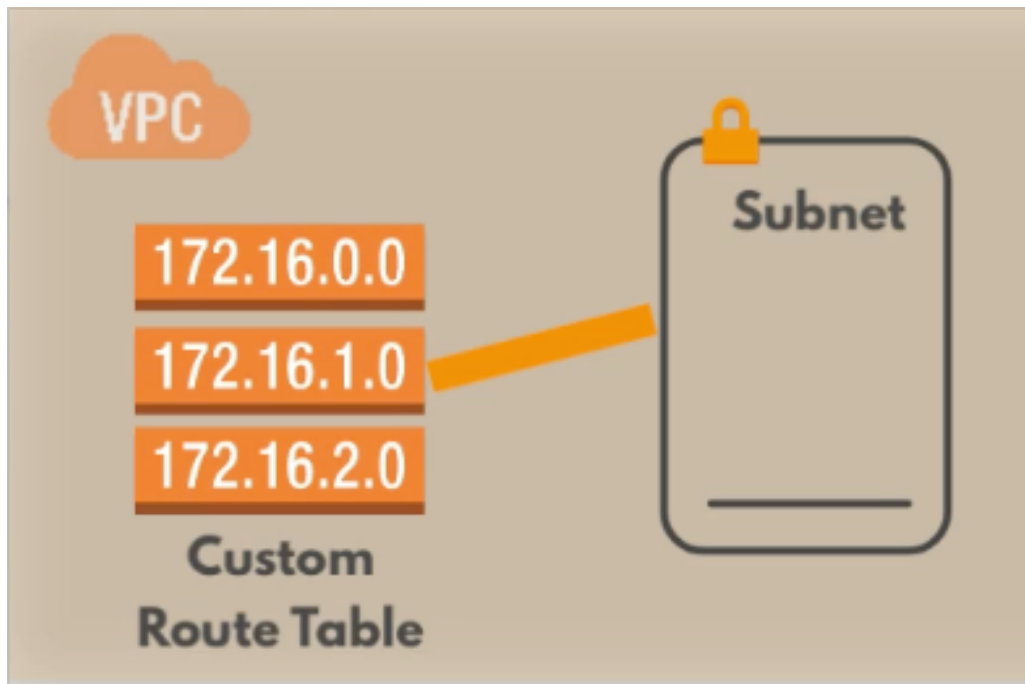
Subnets without explicit associations (1) [Edit sub](#)

The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
dev-subnet-1	subnet-0ca8d536c9baec8db	10.0.10.0/24	-

We need to associate subnets with the route table, so that traffic within the subnet can also be handled by the route table



We can see in the screenshot above that

Explicit subnet associations -> (0)

And that

Subnets without explicit associations -> (1)

Meaning we don't have explicit associations so automatically the subnet got assigned to the MAIN Route table

The screenshot shows the AWS Management Console interface for Route Tables and Subnets. A diagram overlay shows a VPC with a Subnet and a MAIN Route Table. A text box states: "Subnets get automatically assigned to the MAIN Route Table!".

Route tables (1/3)

Name	Route table ID
dev-rtb	rtb-040511274a610
-	rtb-0667162bc1360
-	rtb-0dd6ea9140955

Subnets without explicit associations (3)

The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
-	subnet-060c92eb85ae1de1e	172.31.32.0/20	-

And if we look at our dev-rtb our subnet is also associated by default to the MAIN route table of dev-vpc and the MAIN route table doesn't have IGW

Route tables (1/3) Info Last updated 23 minutes ago Actions Create route table

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge a...	Main	VPC	Owner ID
-	rtb-0c742e93dd4cc284	-	-	Yes	vpc-0635fc1809b770977 dev-vpc	706976333940
☒ dev-rtb	rtb-09e7c96e7e60d6807	-	-	No	vpc-0635fc1809b770977 dev-vpc	706976333940
☐ -	rtb-0118aea5c82fbr9ff	-	-	Yes	vpc-03677c88f7cc2ff15	706976333940

rtb-09e7c96e7e60d6807 / dev-rtb

Details | Routes | **Subnet associations** | Edge associations | Route propagation | Tags

Explicit subnet associations (0) Edit subnet association

Find subnet association

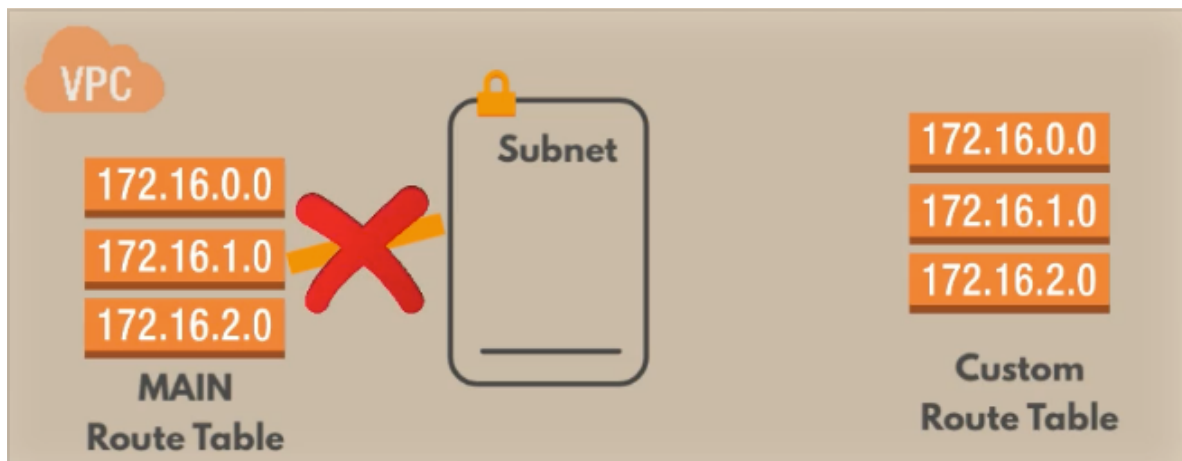
Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
No subnet associations You do not have any subnet associations.			

Subnets without explicit associations (1) Edit subnet association

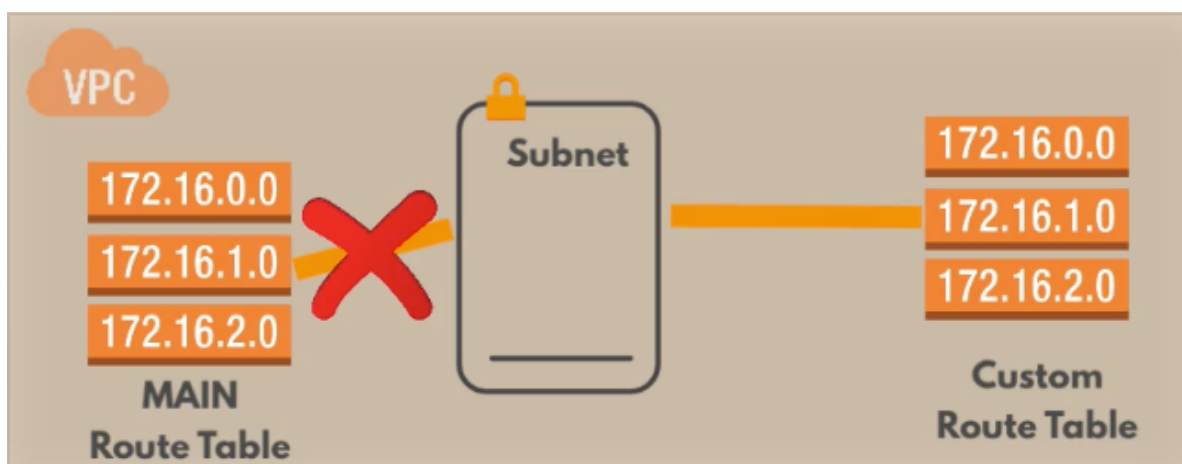
The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
dev-subnet-1	subnet-0ca8d336c9baec8db	10.0.10.0/24	-



This is what we want



We will do the subnet associations in terraform

```

45 resource "aws_route_table_association" "a-rtb-subnet" {
46     subnet_id = aws_subnet.myapp-subnet-1.id
47     route_table_id = aws_route_table.myapp-route-table.id
48 }
49 | You, 37 minutes ago • Uncommitted changes

```

And I associate the specific subnet with the specific route table

-> terraform apply -auto-approve

```

Plan: 1 to add, 0 to change, 0 to destroy.
aws_route_table_association.a-rtb-subnet: Creating...
aws_route_table_association.a-rtb-subnet: Creation complete after 0s [id=rtbassoc-052aae194a0448a6c]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
> terraform git:(Module_12/Terraform_chapter-12) x

```

And we check the AWS UI

Find route tables by attribute or tag

☒	Name	Route table ID	Explicit subnet associ...	Edge a...	Main	VPC	Owner ID
☐	-	rtb-0c742e93dd4ccc284	-	-	Yes	vpc-06355fc1809b770977 dev-vpc	706976333940
☒	dev-rtb	rtb-09e7c96e7e60d6807	subnet-0ca8d336c9baec...	-	No	vpc-06355fc1809b770977 dev-vpc	706976333940
☐	-	rtb-0118aea5c82fbf9ff	-	-	Yes	vpc-03677c88f7cc27f15	706976333940

rtb-09e7c96e7e60d6807 / dev-rtb

Details

Routes

Subnet associations

Edge associations

Route propagation

Tags

Explicit subnet associations (1)

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
dev-subnet-1	subnet-0ca8d336c9baec8db	10.0.10.0/24	-

Subnets without explicit associations (0)

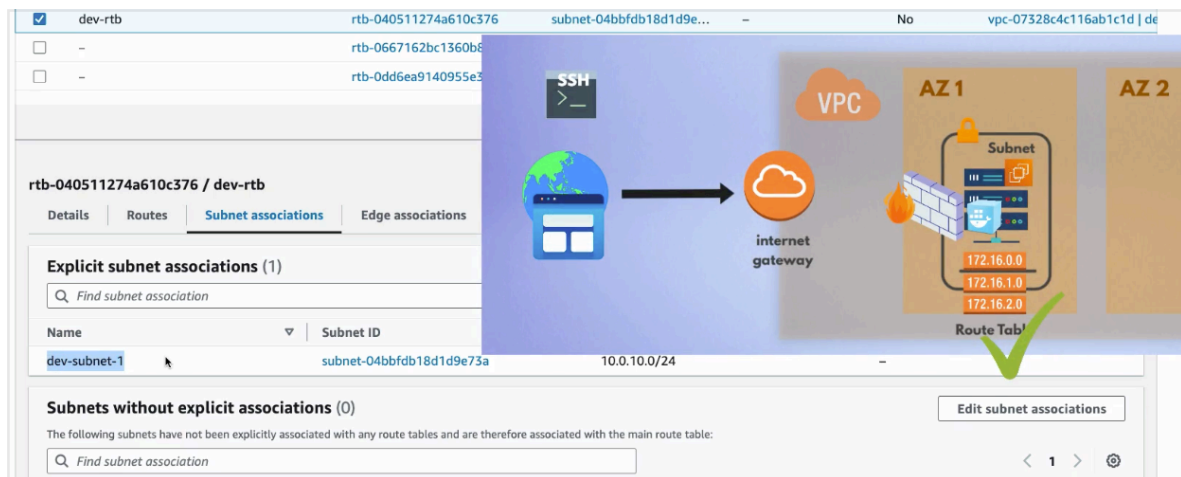
The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
------	-----------	-----------	-----------

No subnets without explicit associations
All your subnets are associated with a route table.

And the explicit association is done!



Use the Main Route Table

If we prefer to use the default route table of our Tf created VPC then we remove the resources: 'aws_route_table' and 'aws_route_table_association'

```

24
25
26 #resource "aws_route_table" "myapp-route-table" {
27     # vpc_id = aws_vpc.myapp-vpc.id
28     #
29     # route {
30     #     cidr_block = "0.0.0.0/0"
31     #     gateway_id = aws_internet_gateway.myapp-igw.id
32     # }
33     # tags = {
34     #     Name = "${var.env_prefix}-rtb"
35     # }
36 #}
37
38 resource "aws_internet_gateway" "myapp-igw" {
39     vpc_id = aws_vpc.myapp-vpc.id
40     tags = {
41         Name = "${var.env_prefix}-igw"
42     }
43 }
44
45 #resource "aws_route_table_association" "a-rtb-subnet" {
46     # subnet_id = aws_subnet.myapp-subnet-1.id
47     # route_table_id = aws_route_table.myapp-route-table.id
48 #}
49

```

-> terraform apply -auto-approve

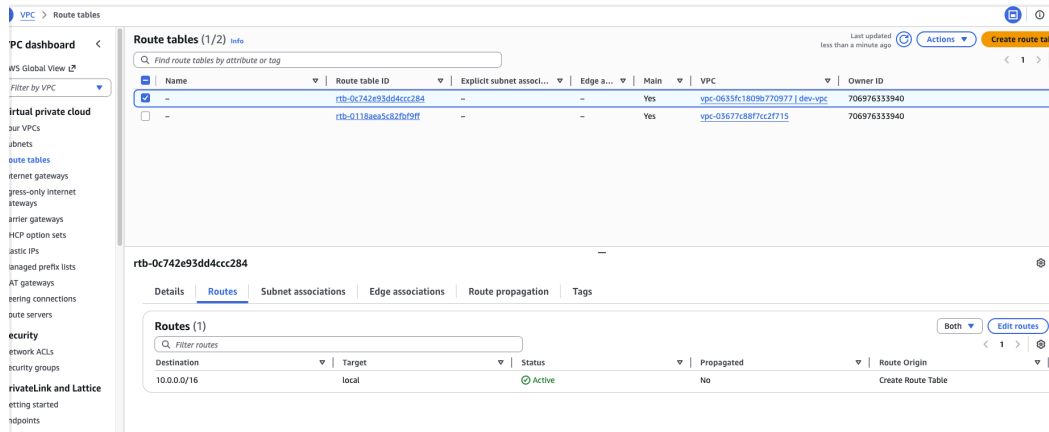
```

Plan: 0 to add, 0 to change, 2 to destroy.
aws_route_table_association.a-rtb-subnet: Destroying... [id=rtbassoc-052aae194a0448a6c]
aws_route_table_association.a-rtb-subnet: Destruction complete after 0s
aws_route_table.myapp-route-table: Destroying... [id=rtb-09e7c96e7e60d6807]
aws_route_table.myapp-route-table: Destruction complete after 1s

Apply complete! Resources: 0 added, 0 changed, 2 destroyed.

```

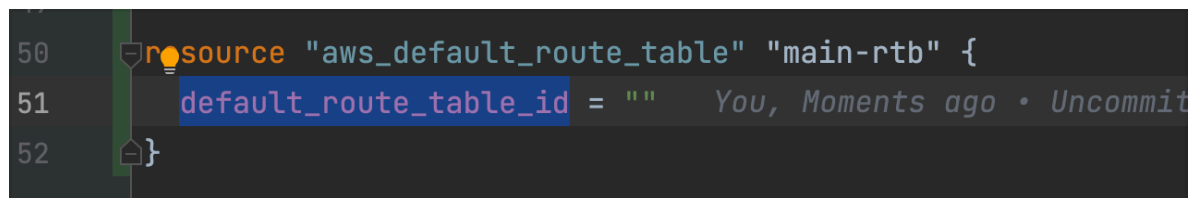
So now we only have the VPC's default route table



And then we edit the default route table



And it doesn't need the attribute 'vpc_id' because the route table already exists instead we need the id of the existing route table:



Because I already run terraform apply before and the route table already exist then I can get the value with
 -> terraform state show aws_vpc.myapp-vpc


```

→ terraform git:(Module_12/Terraform_chapter-12) × terraform state show aws_vpc.myapp-vpc
# aws_vpc.myapp-vpc:
resource "aws_vpc" "myapp-vpc" {
  arn                                = "arn:aws:ec2:eu-west-2:706976333940:vpc/vpc-0635fc1809b770977"
  assign_generated_ipv6_cidr_block   = false
  cidr_block                         = "10.0.0.0/16"
  default_network_acl_id            = "acl-0f23e73e559ecf842"
  default_route_table_id            = "rtb-0c742e93dd4ccc284"
  default_security_group_id         = "sg-0a5cd9a9bdd635975"
  dhcp_options_id                   = "dopt-071348549901a183c"
  enable_dns_hostnames              = false
  enable_dns_support                 = true
  enable_network_address_usage_metrics = false
}

```

So I get the value from there

```

50 resource "aws_default_route_table" "main-rtb" {
51   default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
52 }

```

And I add the other attributes I used when creating the new route table

```

50 resource "aws_default_route_table" "main-rtb" {
51   default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
52
53   route {
54     cidr_block = "0.0.0.0/0"
55     gateway_id = aws_internet_gateway.myapp-igw.id
56   }
57   tags = {
58     Name: "${var.env_prefix}-rtb"
59   }
60 }

```

And edit the tag:

```

50 resource "aws_default_route_table" "main-rtb" {
51   default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
52
53   route {
54     cidr_block = "0.0.0.0/0"
55     gateway_id = aws_internet_gateway.myapp-igw.id
56   }
57   tags = {
58     Name: "${var.env_prefix}-main-rtb"
59   }
60 }

```

-> terraform apply -auto-approve

```
Plan: 1 to add, 0 to change, 0 to destroy.
aws_default_route_table.main-rtb: Creating...
aws_default_route_table.main-rtb: Creation complete after 0s [id=rtb-0c742e93dd4ccc284]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

So now it has the IGW

The screenshot shows the AWS Management Console interface for Route Tables. The 'dev-main-rtb' route table is selected, and the 'Routes' tab is active. It displays two routes:

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-00c30f6b1602ed06f	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

But we have not defined explicit subnet associations, and we don't need to -> by default the subnets get associated to the default/main route table and we are using the default route table

The screenshot shows the AWS Management Console interface for Route Tables. The 'dev-main-rtb' route table is selected, and the 'Subnet associations' tab is active. It displays 'Explicit subnet associations (0)' and 'Subnets without explicit associations (1)'. The subnets without explicit associations are listed as 'dev-subnet-1' with CIDR '10.0.10.0/24'.

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
dev-subnet-1	subnet-0ca8d336c9baec8db	10.0.10.0/24	-

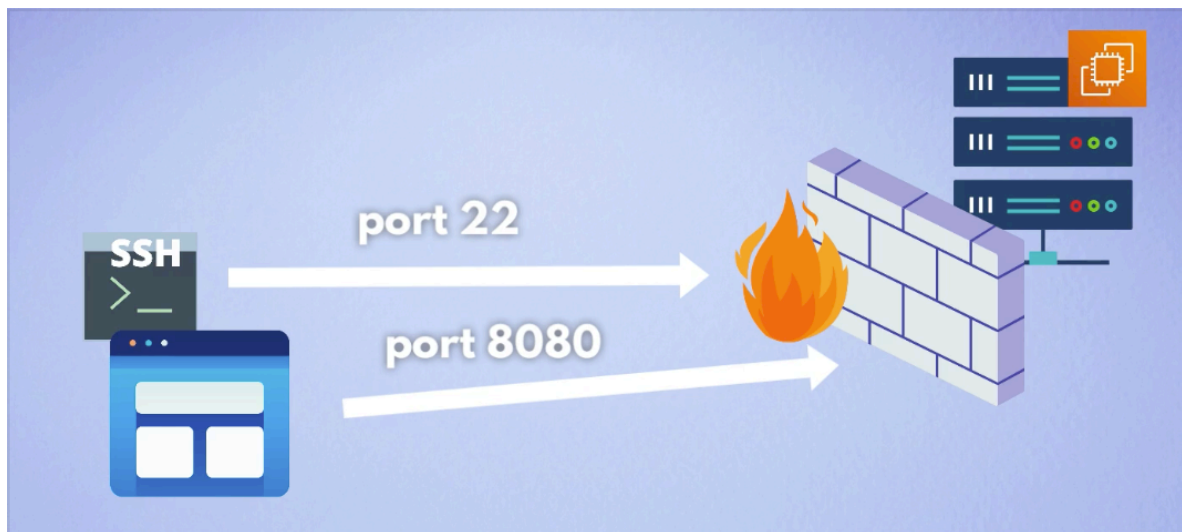
Security Group

6. Create Security group -> firewall.

Configure firewall rules for our EC2 instance.

We want to be able to ssh into our EC2 instance -> open port 22

We want to access the nginx web server that we deployed as a container via the web browser -> open port 8080



Let's create a security group

```
62 resource "aws_security_group" "myapp-sg" {
63     name = "myapp-sg"
64     vpc_id = aws_vpc.myapp-vpc.id
65
66     | You, A minute ago • Uncommitted changes
67 }
```

— tangent —

Q:

why when creating a security group

```
resource "aws_security_group" "myapp-sg" {
    name = "myapp-sg"
    vpc_id = aws_vpc.myapp-vpc.id

}
```

I need the vpc ID but I don't add the subnet id?

A:

- It's because of how AWS networking layers work:

- Security Groups are attached at the VPC level — they need to know which VPC they belong to, but they're not tied to a specific subnet
- Subnets are attached to EC2 instances — when you create an EC2 instance, you specify which subnet it goes in

So the flow is:

1. VPC — the overall network
2. Subnet — a segment inside the VPC
3. Security Group — a firewall rule set that belongs to the VPC
4. EC2 instance — goes into a specific subnet AND has a security group attached to it

The Security Group and Subnet come together at the EC2 instance level, not before. When you create your EC2 resource in Terraform you'll have something like:

```
resource "aws_instance" "myapp-server" {  
  subnet_id      = aws_subnet.myapp-subnet.id  
  vpc_security_group_ids = [aws_security_group.myapp-sg.id]  
}
```

That's where both get connected. The security group just needs to know the VPC so AWS knows its boundaries.

— end of tangent —

Now let's define the firewall rules of the security group:

Generally we have two types of rules:

1. traffic that is coming inside the VPC and entering the servers -> incoming traffic

Firewall Rules

**incoming
traffic**

- SSH into EC2
- access from browser

We use an attribute Called 'ingress' to set the incoming traffic config

```
62 resource "aws_security_group" "myapp-sg" {  
63     name = "myapp-sg"  
64     vpc_id = aws_vpc.myapp-vpc.id  
65  
66     ingress {  
67         from_port = 22  
68         to_port = 22  
69     }  
70 }
```

We can set a range of ports that is why we have from - to but in our case we only one one port. 🙌

Then we add the protocol which is TCP

```

66 ingress {
67     from_port = 22
68     to_port = 22
69     protocol = "TCP"
70 }
71 }

```

And then we add the 'source' -> who is allowed to access the port range which in our case is only 22, which IP addresses are allowed

```

66 ingress {
67     from_port = 22
68     to_port = 22
69     protocol = "TCP"
70     cidr_blocks = []
71 }
72 }

```

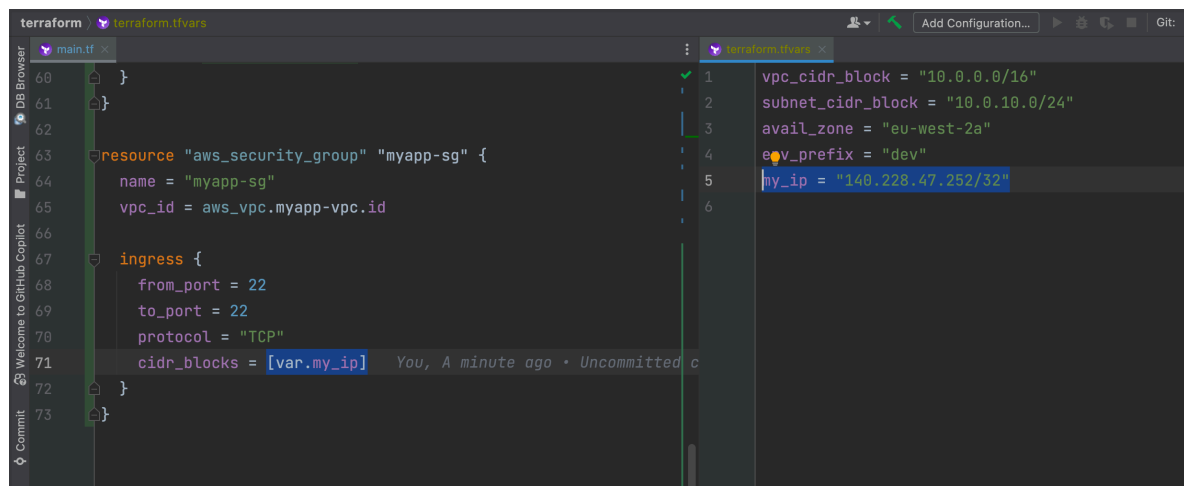
And we add a list of trusted IP addresses. In our case it will be only one item -> our local machine IP address. So we will fixate at /32

```

66 ingress {
67     from_port = 22
68     to_port = 22
69     protocol = "TCP"
70     cidr_blocks = [140.228.47.252/32]
71 }
72 }

```

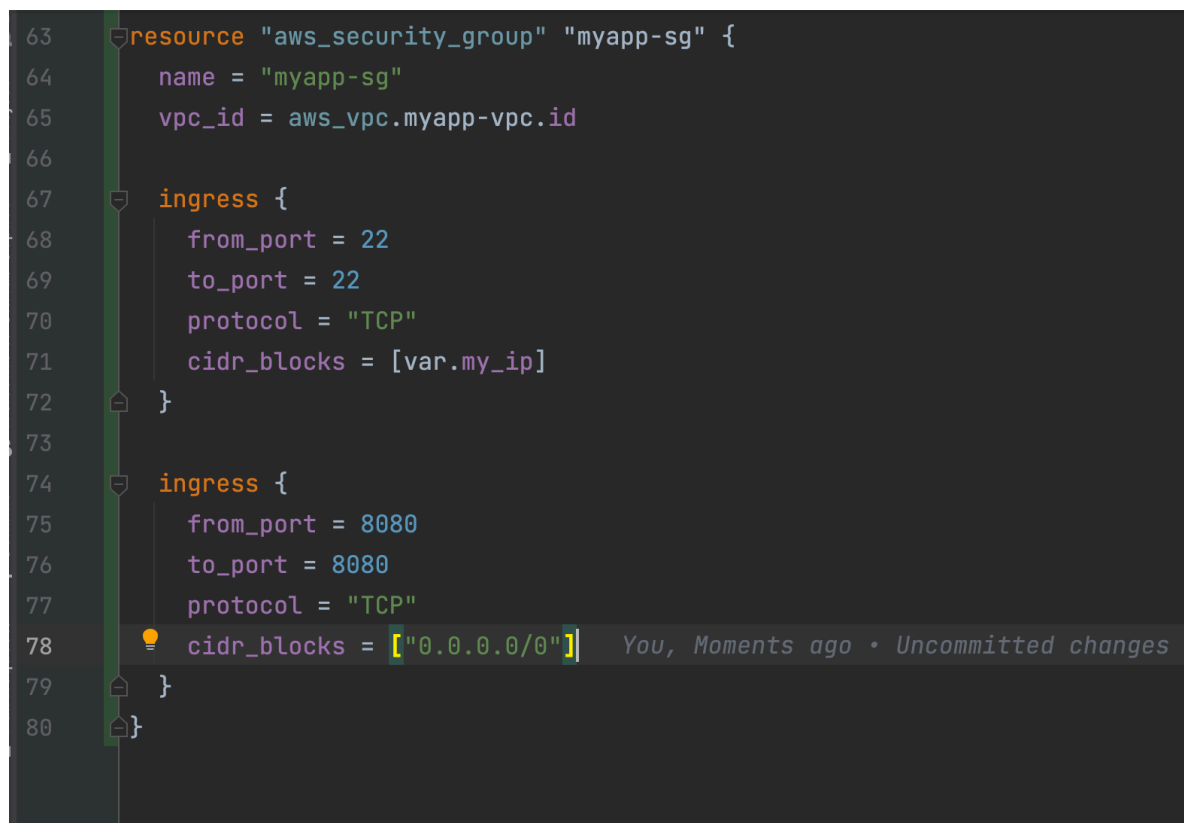
This value can be turn into a variable instead of being hardcoded 🙏



```
main.tf
60 }
61 }
62
63 resource "aws_security_group" "myapp-sg" {
64     name = "myapp-sg"
65     vpc_id = aws_vpc.myapp-vpc.id
66
67     ingress {
68         from_port = 22
69         to_port = 22
70         protocol = "TCP"
71         cidr_blocks = [var.my_ip]
72     }
73 }
```

```
terraform.tfvars
1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 avail_zone = "eu-west-2a"
4 vpc_prefix = "dev"
5 my_ip = "140.228.47.252/32"
6
```

Now we will create a second ingress rule to config port 8080 and anyone can access



```
63 resource "aws_security_group" "myapp-sg" {
64     name = "myapp-sg"
65     vpc_id = aws_vpc.myapp-vpc.id
66
67     ingress {
68         from_port = 22
69         to_port = 22
70         protocol = "TCP"
71         cidr_blocks = [var.my_ip]
72     }
73
74     ingress {
75         from_port = 8080
76         to_port = 8080
77         protocol = "TCP"
78         cidr_blocks = ["0.0.0.0/0"]
79     }
80 }
```

2. Traffic going outside the VPC

Firewall Rules

incoming traffic

- SSH into EC2
- access from browser

outgoing traffic

- installations
- fetch Docker Image

We use an attribute called 'Egress'

When our app needs to fetch something from internet to install other software or it needs to fetch a docker image for example it will use different IP addresses so we want this to be any 0.0.0.0/0 and we don't want to limit the protocols either

```
81  egress {  
82      from_port = 0  
83      to_port   = 0  
84      protocol  = "-1"  
85      cidr_blocks = ["0.0.0.0/0"]  
86      prefix_list_ids = []  
87  }  
88 }
```

And we also add an empty 'prefix_list_ids' if we add values we will allow access to VPC endpoints

— tangent —

prefix_list_ids = [] is just an empty list — it means you're not using any VPC endpoints, so it has no real effect here.

What `prefix_list_ids` actually is:

A prefix list is a managed list of IP ranges for AWS services. For example, instead of knowing all the IP ranges for S3, AWS gives you a prefix list ID like `pl-xxxxxxx` that represents all S3 IPs in that region.

You'd use it when you want to allow traffic to an AWS service via a VPC endpoint without going through the internet:

```
egress {  
  from_port    = 0  
  to_port      = 0  
  protocol     = "-1"  
  prefix_list_ids = ["pl-xxxxxxx"] # e.g. S3 VPC endpoint  
}
```

In your code:

Since you already have `cidr_blocks = ["0.0.0.0/0"]` which allows all outbound traffic to everywhere, the empty `prefix_list_ids = []` is redundant — it's not doing anything. Your teacher may have added it just to show the field exists.

So the explanation "to allow access to VPC endpoints" is a bit misleading — an empty list allows nothing. It would only matter if you put actual prefix list IDs in there.

— end of tangent —

And finally add the tags

```
74 ingress {
75     from_port = 8080
76     to_port   = 8080
77     protocol  = "TCP"
78     cidr_blocks = ["0.0.0.0/0"]
79 }
80
81 egress {
82     from_port = 0
83     to_port   = 0
84     protocol  = "-1"
85     cidr_blocks = ["0.0.0.0/0"]
86     prefix_list_ids = []
87 }
88
89 tags = {
90     Name: "${var.env_prefix}-sg" You, 5 minutes ago
91 }
92 }
```

-> terraform plan

```

➔ terraform git:(Module_12/Terraform_chapter-12) ✕ terraform plan
aws_vpc.myapp-vpc: Refreshing state... [id=vpc-0635fc1809b770977]
aws_internet_gateway.myapp-igw: Refreshing state... [id=igw-00c30f6b1602ed06f]
aws_subnet.myapp-subnet-1: Refreshing state... [id=subnet-0ca8d336c9baec8db]
aws_default_route_table.main-rtb: Refreshing state... [id=rtb-0c742e93dd4ccc284]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_security_group.myapp-sg will be created
+ resource "aws_security_group" "myapp-sg" {
+   arn                = (known after apply)
+   description         = "Managed by Terraform"
+   egress              = [
+     {
+       cidr_blocks      = [
+         "0.0.0.0/0",

```

-> terraform apply -auto-approve

```

Plan: 1 to add, 0 to change, 0 to destroy.
aws_security_group.myapp-sg: Creating...
aws_security_group.myapp-sg: Creation complete after 3s [id=sg-0eb8a0c1ef3e7aed6]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
➔ terraform git:(Module_12/Terraform_chapter-12) ✕

```

We check our AWS UI

Name	Security group ID	Security group name	VPC ID	Description
dev-sg	sg-0eb8a0c1ef3e7aed6	myapp-sg	vpc-0635fc1809b770977	Managed by Terraform
-	sg-0638fd0a242d48af6	default	vpc-03677c88f7cc2f715	default VPC security group
-	sg-0a5cd9a9b0dd635975	default	vpc-0635fc1809b770977	default VPC security group
-	sg-087818984cfad7e94	my-sg	vpc-03677c88f7cc2f715	My SG
-	sg-09e272e3b7cc7f90b	security-group-docker-server	vpc-03677c88f7cc2f715	launch-wizard-1 created 2026-03-24T14:19:13.241Z

We have two security groups:

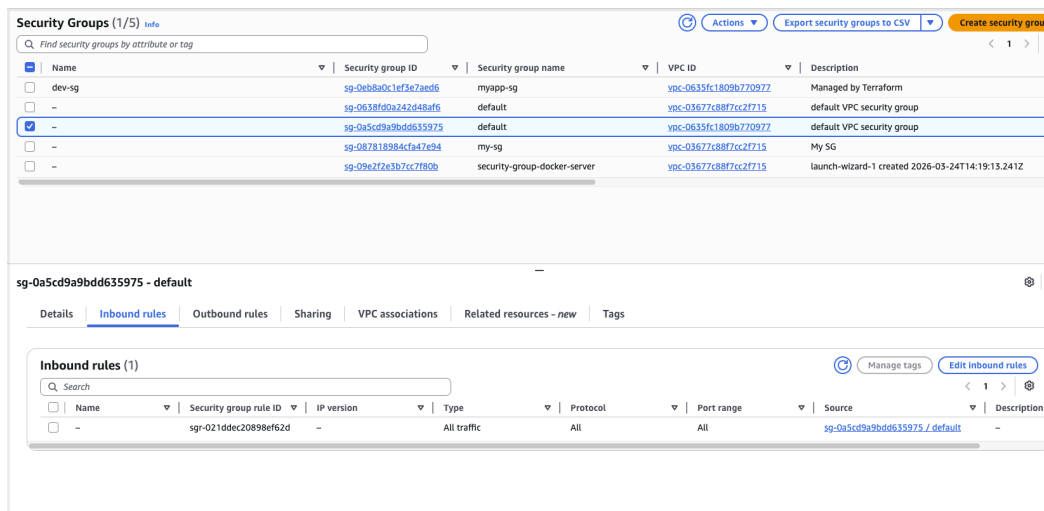
- Default sg created with our TF VPC
- New sg we created for our TF VPC

```

11 resource "aws_vpc" "myapp-vpc" {
12     cidr_block = var.vpc_cidr_block
13     tags = {
14         Name: "${var.env_prefix}-vpc"
15     }
16 }

```

And the default sg is blocked for external traffic



Name	Security group ID	Security group name	VPC ID	Description
dev-sg	sg-0eb8a0c1ef3e7aed6	myapp-sg	vpc-0635fc1809b770977	Managed by Terraform
-	sg-0638f9da242d48af6	default	vpc-03677c88f7cc2f715	default VPC security group
-	sg-0a5cd9a9bdd635975	default	vpc-0635fc1809b770977	default VPC security group
-	sg-087818984cf47e94	my-sg	vpc-03677c88f7cc2f715	My SG
-	sg-09e2f2e3b7cc7f90b	security-group-docker-server	vpc-03677c88f7cc2f715	launch-wizard-1 created 2026-03-24T14:19:13.241Z

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-021ddc20898ef62d	-	All traffic	All	All	sg-0a5cd9a9bdd635975 / default	-

Note

Source: sg-0a5cd9a9bdd635975 / default

This means it allows all traffic from any resource that belongs to the same security group. This is a self-referencing rule — resources within the same security group can talk to each other freely.

How to know if it's truly closed:

A closed security group would have no inbound rules at all — an empty inbound table.

What you have:

- Inbound: allows all traffic from resources in the same SG (self-reference)
- This is actually AWS's default behavior for the default security group — AWS adds this rule automatically

Default SG vs custom SG:

- Default security group (the one AWS creates automatically with every VPC)
- comes with this self-referencing rule

- Custom security group (one you create yourself) — starts with no inbound rules, fully closed

Since your Terraform code creates a custom `aws_security_group` without defining any inbound rules, your custom security group is closed for inbound. What you're seeing in the console is likely the default SG AWS created automatically, not your custom one.

Q:

so it is blocked for external traffic but open for internal traffic that share the sg

A:

Exactly, that's a perfect way to put it:

- External traffic — blocked, nothing from outside can get in
- Internal traffic — open, any resource sharing the same security group can communicate freely with each other

This is useful for example if you have multiple EC2 instances in the same security group — they can talk to each other without needing to open specific ports to the outside world.

Let's look at our SG and its inbound rules:

The screenshot shows the AWS Management Console interface. At the top, there's a 'Security Groups (1/5)' header with a search bar and buttons for 'Actions', 'Export security groups to CSV', and 'Create security group'. Below this is a table listing security groups:

Name	Security group ID	Security group name	VPC ID	Description
dev-sg	sg-0eb8a0c1ef3e7aed6	myapp-sg	vpc-0635fc1809b770977	Managed by Terraform
-	sg-0638f0a242d48af6	default	vpc-03677c887cc27715	default VPC security group
-	sg-0a5cd9a9bd6d635975	default	vpc-0635fc1809b770977	default VPC security group
-	sg-087818984cfa47e94	my-sg	vpc-03677c887cc27715	My SG
-	sg-09e27e3b7cc77800	security-group-docker-server	vpc-03677c887cc27715	launch-wizard-1 created 2026-05-24T14:19:13.241Z

Below the table, the 'dev-sg' is selected, and the 'Inbound rules' tab is active. It shows a table of inbound rules:

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sg-06dc5526a66c16643	IPv4	Custom TCP	TCP	8080	0.0.0.0/0	-
-	sg-04e6e7369b863a2a5	IPv4	SSH	TCP	22	140.228.47.252/32	-

And the outbound rule:

Security Groups (1/5) Info

Find security groups by attribute or tag

Name	Security group ID	Security group name	VPC ID	Description
☒ dev-sg	sg-0eb8a0c1ef3e7aed6	myapp-sg	vpc-0635fc1809b770977	Managed by Terraform
☐ -	sg-0638f60a242d48af6	default	vpc-03677c88f7cc2f715	default VPC security group
☐ -	sg-0a5cd9a9bd0635975	default	vpc-0635fc1809b770977	default VPC security group
☐ -	sg-087818984cfa47e94	my-sg	vpc-03677c88f7cc2f715	My SG
☐ -	sg-09e27e3b7cc7f80b	security-group-docker-server	vpc-03677c88f7cc2f715	launch-wizard-1 created 2026-03-24T14:19:13.241Z

sg-0eb8a0c1ef3e7aed6 - myapp-sg

Details Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Outbound rules (1/1)

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range	Destination	Description
☒ -	sg-05385fca73c3cbe5f	IPv4	All traffic	All	All	0.0.0.0/0	-

So now SG is ready to be assigned to an EC2 instance!

We also can use the default SG created with the TF VPC if we prefer.

Use Default Security Group

We create a resource of type "aws_default_security_group" and we edit the name of the resource to "default-sg" and we don't need a name as it already has the name "default". Everything else is the same

```

61 }
62
63 resource "aws_default_security_group" "default-sg" {
64     # name = "myapp-sg"
65     vpc_id = aws_vpc.myapp-vpc.id
66
67     ingress {
68         from_port = 22
69         to_port = 22
70         protocol = "TCP"
71         cidr_blocks = [var.my_ip]
72     }
73
74     ingress {
75         from_port = 8080
76         to_port = 8080
77         protocol = "TCP"
78         cidr_blocks = ["0.0.0.0/0"]
79     }
80
81     egress {
82         from_port = 0
83         to_port = 0
84         protocol = "-1"
85         cidr_blocks = ["0.0.0.0/0"]

```

And we edit the tag name to 'default-sg' to be more descriptive

```

80
81     egress {
82         from_port = 0
83         to_port   = 0
84         protocol  = "-1"
85         cidr_blocks = ["0.0.0.0/0"]
86         prefix_list_ids = []
87     }
88
89     tags = {
90         Name: "${var.env_prefix}-default-sg"
91     }
92 }

```

-> terraform plan

```

+ terraform git:(Module_12/Terraform_chapter-12) x terraform plan
aws_security_group.myapp-sg: Refreshing state... [id=sg-0eb8a0c1ef3e7aed6]
aws_vpc.myapp-vpc: Refreshing state... [id=vpc-0635fc1809b770977]
aws_internet_gateway.myapp-igw: Refreshing state... [id=igw-00c30f6b1602ed06f]
aws_subnet.myapp-subnet-1: Refreshing state... [id=subnet-0ca8d336c9baec8db]
aws_default_route_table.main-rtb: Refreshing state... [id=rtb-0c742e93dd4ccc284]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create
- destroy

Terraform will perform the following actions:

# aws_default_security_group.default-sg will be created
+ resource "aws_default_security_group" "default-sg" {
+   arn                = (known after apply)
+   description        = (known after apply)
+   egress              = [
+     {
+       cidr_blocks     = [
+         "0.0.0.0/0",

```

One resource will be created

One resource will be destroyed

-> terraform apply -auto-approve

```

Plan: 1 to add, 0 to change, 1 to destroy.
aws_security_group.myapp-sg: Destroying... [id=sg-0eb8a0c1ef3e7aed6]
aws_default_security_group.default-sg: Creating...
aws_security_group.myapp-sg: Destruction complete after 1s
aws_default_security_group.default-sg: Creation complete after 2s [id=sg-0a5cd9a9bdd635975]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
→ terraform git:(Module_12/Terraform_chapter-12) x

```

Now check our AWS UI

The screenshot shows the AWS Management Console interface for Security Groups. At the top, there's a search bar and buttons for 'Actions', 'Export security groups to CSV', and 'Create security group'. Below this is a table of security groups. The 'dev-default-sg' is selected, and its details are shown below. The 'Inbound rules' tab is active, showing two rules. The first rule is for port 22 (SSH) and the second rule is for port 8080 (Custom TCP). The security group name is 'default'.

Name	Security group ID	Security group name	VPC ID	Description
-	sg-0638f0a242d48af6	default	vpc-03677c8877cc2f715	default VPC security group
-	sg-087818984cf47e94	my-sg	vpc-03677c8877cc2f715	My SG
-	sg-09e272e5b7cc7f90b	security-group-docker-server	vpc-03677c8877cc2f715	launch-wizard-1 created 2026-03-24T14:19:13.241Z
dev-default-sg	sg-0a5cd9a9bdd635975	default	vpc-0635fc1809b770977	default VPC security group

sg-0a5cd9a9bdd635975 - default

Details | **Inbound rules** | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (2)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-02da7104a98cd619f	IPv4	SSH	TCP	22	140.228.47.252/32	-
-	sgr-0fb9d4fa0fac6d6e	IPv4	Custom TCP	TCP	8080	0.0.0.0/0	-

And we can see that the inbound rules have our custom configuration and the security group name is still 'default'

Next chapter (Module 12 - chapter 13)

4. Then we will create the EC2 instance
5. Deploy docker container for nginx